Announcement

Since Monday the 5th of May is a bank Holiday, next weeks lectures are scheduled as follows.

- Tuesday 6th of May, 16:00-17:00 Conference Auditorium 1
- Wednesday 7th of May, 9:00-10:00 RBLT
- Tutorials are also moved to Tuesday or Thursday $\longrightarrow$ please consult your timetable for time and location!

UNIVERSITY OF LEEDS

Formal Language & Finite Automata

Summary

Dr Noleen Koehler

University of Leeds

Overview

1. Regular languages
2. Context-free languages
3. Turing machines

Regular languages

DFA's

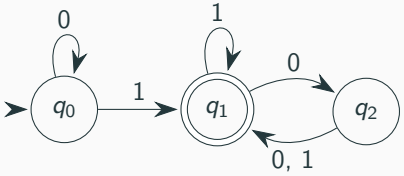
Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.



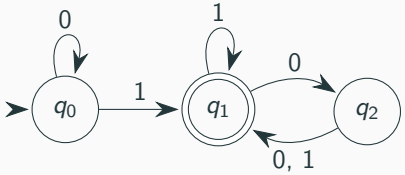
DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.



DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

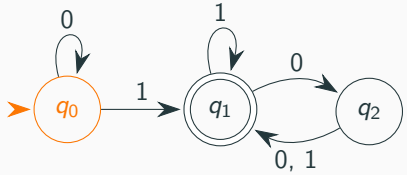
- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.

0 1 0 0

Sequence of states: q_0,



DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

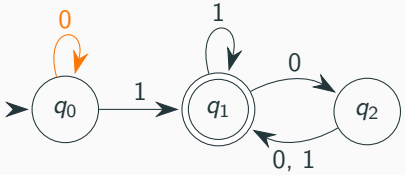
- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.

0 1 0 0

Sequence of states: q_0,



DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

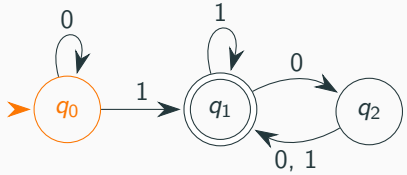
- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.

0 1 0 0

Sequence of states: q_0, q_0,



DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

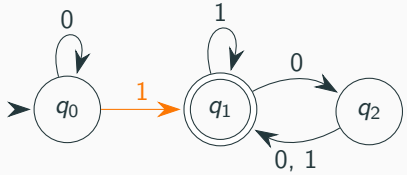
- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.

0 1 0 0

Sequence of states: q_0, q_0,



DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

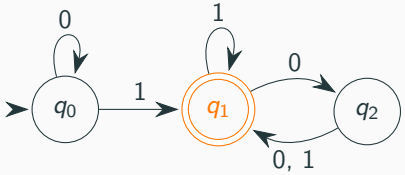
- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.

0 1 0 0

Sequence of states: q_0, q_0, q_1,



DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

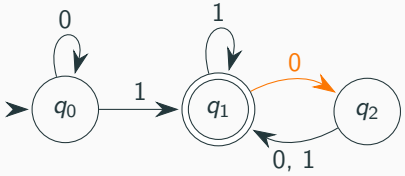
- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.

0 1 0 0

Sequence of states: q_0, q_0, q_1,



DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

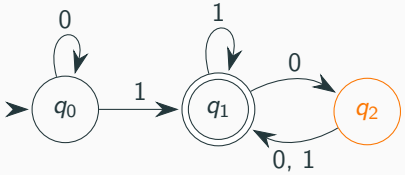
- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.

0 1 0 0

Sequence of states: q_0, q_0, q_1, q_2,



DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

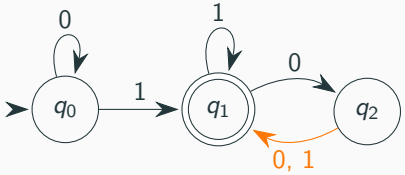
- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.

0 1 0 0

Sequence of states: q_0, q_0, q_1, q_2,



DFA's

Definition (DFA)
A **deterministic finite automaton** M is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

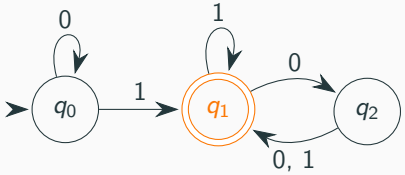
- Q is a finite set called the **states**,
- Σ is a finite set called the **alphabet**,
- $\delta : Q \times \Sigma \to Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of a DFA)
The DFA M **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ exists such that

- $r_0 = q_0$,
- $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i < n$, and
- $r_n \in F$.

0 1 0 0

Sequence of states: q_0, q_0, q_1, q_2, q_1



NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where

-Q is a finite set called the **states**,

-Σ is a finite set called the **alphabet**,

-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,

-$q_0 \in Q$ is the **start state**, and

-$F \subseteq Q$ is the set of **accepting states**.

NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where

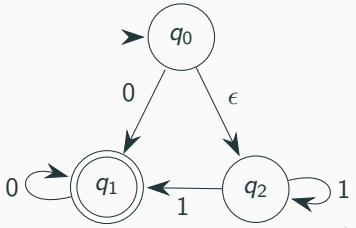
-Q is a finite set called the **states**,

-Σ is a finite set called the **alphabet**,

-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,

-$q_0 \in Q$ is the **start state**, and

-$F \subseteq Q$ is the set of **accepting states**.



NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where

-Q is a finite set called the **states**,

-Σ is a finite set called the **alphabet**,

-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,

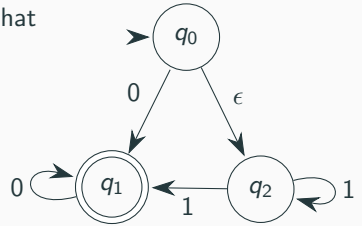
-$q_0 \in Q$ is the **start state**, and

-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that

-$r_0 = q_0$,

-$r_{i+1} \in \delta(r_i, w_{i+1})$,



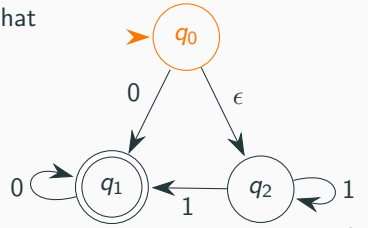
NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where
-Q is a finite set called the **states**,
-Σ is a finite set called the **alphabet**,
-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,
-$q_0 \in Q$ is the **start state**, and
-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that
-$r_0 = q_0$,
-$r_{i+1} \in \delta(r_i, w_{i+1})$, for $0 \leq i \leq n-1$, and
-$r_n \in F$.

1 1 1

Sequence of states: q_0



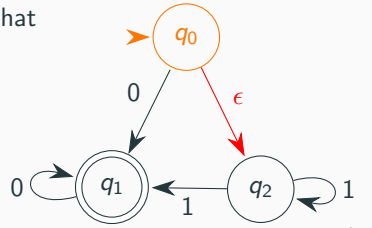
NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where
-Q is a finite set called the **states**,
-Σ is a finite set called the **alphabet**,
-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,
-$q_0 \in Q$ is the **start state**, and
-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that
-$r_0 = q_0$,
-$r_{i+1} \in \delta(r_i, w_{i+1})$,
for $0 \leq i \leq n-1$, and
-$r_n \in F$.

ϵ 1 1 1

Sequence of states: q_0



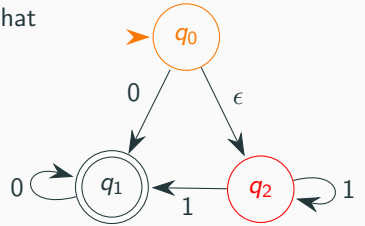
NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where
-Q is a finite set called the **states**,
-Σ is a finite set called the **alphabet**,
-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,
-$q_0 \in Q$ is the **start state**, and
-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that
-$r_0 = q_0$,
-$r_{i+1} \in \delta(r_i, w_{i+1})$,
for $0 \leq i \leq n-1$, and
-$r_n \in F$.

$\epsilon \quad 1 \quad 1 \quad 1$

Sequence of states:



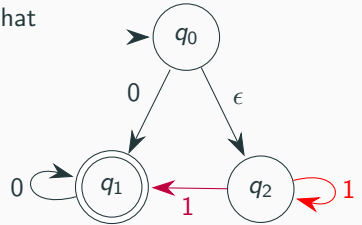
NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where
-Q is a finite set called the **states**,
-Σ is a finite set called the **alphabet**,
-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,
-$q_0 \in Q$ is the **start state**, and
-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that
-$r_0 = q_0$,
-$r_{i+1} \in \delta(r_i, w_{i+1})$,
for $0 \leq i \leq n-1$, and
-$r_n \in F$.

ϵ 1 1 1

Sequence of states:



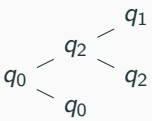
NFA's

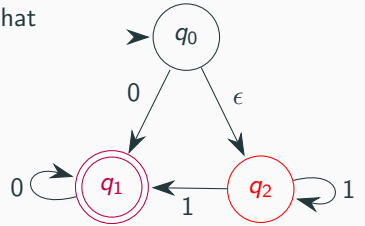
Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where
-Q is a finite set called the **states**,
-Σ is a finite set called the **alphabet**,
-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,
-$q_0 \in Q$ is the **start state**, and
-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that
-$r_0 = q_0$,
-$r_{i+1} \in \delta(r_i, w_{i+1})$,
for $0 \leq i \leq n-1$, and
-$r_n \in F$.

ϵ 1 1 1

Sequence of states:





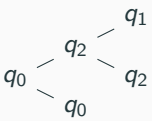
NFA's

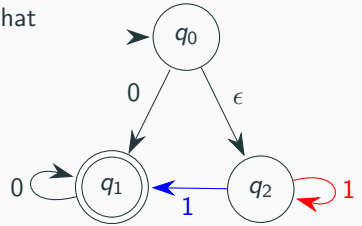
Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where
-Q is a finite set called the **states**,
-Σ is a finite set called the **alphabet**,
-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,
-$q_0 \in Q$ is the **start state**, and
-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that
-$r_0 = q_0$,
-$r_{i+1} \in \delta(r_i, w_{i+1})$,
for $0 \leq i \leq n-1$, and
-$r_n \in F$.

ϵ 1 1 1

Sequence of states:





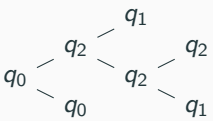
NFA's

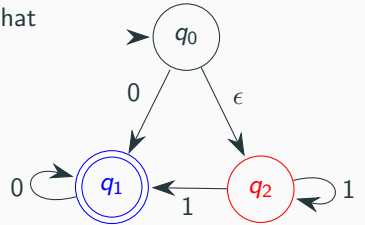
Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where
-Q is a finite set called the **states**,
-Σ is a finite set called the **alphabet**,
-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,
-$q_0 \in Q$ is the **start state**, and
-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that
-$r_0 = q_0$,
-$r_{i+1} \in \delta(r_i, w_{i+1})$,
for $0 \leq i \leq n-1$, and
-$r_n \in F$.

ϵ 1 1 1

Sequence of states:



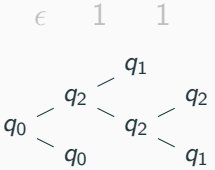


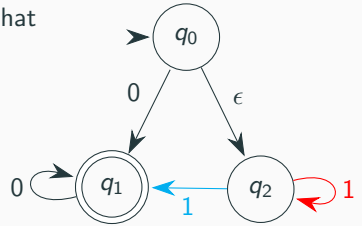
NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where
-Q is a finite set called the **states**,
-Σ is a finite set called the **alphabet**,
-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,
-$q_0 \in Q$ is the **start state**, and
-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that
-$r_0 = q_0$,
-$r_{i+1} \in \delta(r_i, w_{i+1})$,
for $0 \leq i \leq n-1$, and
-$r_n \in F$.

Sequence of states:



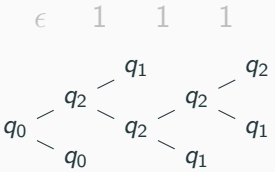


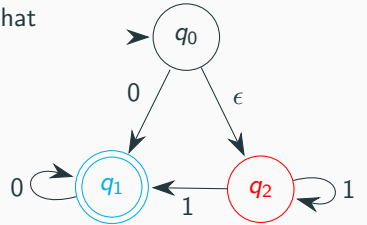
NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where
-Q is a finite set called the **states**,
-Σ is a finite set called the **alphabet**,
-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,
-$q_0 \in Q$ is the **start state**, and
-$F \subseteq Q$ is the set of **accepting states**.

Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that
-$r_0 = q_0$,
-$r_{i+1} \in \delta(r_i, w_{i+1})$,
for $0 \leq i \leq n-1$, and
-$r_n \in F$.

Sequence of states:





NFA's

Definition (NFA)
A **non-deterministic finite automaton** is a 5-tuple $N = (Q, \Sigma, \delta, q_0, F)$, where

-Q is a finite set called the **states**,

-Σ is a finite set called the **alphabet**,

-$\delta : Q \times \Sigma_\epsilon \to \mathcal{P}(Q)$ is the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$,

-$q_0 \in Q$ is the **start state**, and

-$F \subseteq Q$ is the set of **accepting states**.

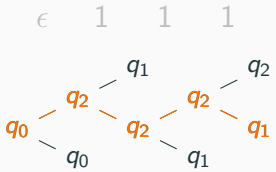
Definition (Computation of an NFA)
The NFA N accepts w if we can write $w = y_1, \ldots, y_n$ where $y_i \in \Sigma_\epsilon$ for $1 \leq i \leq n$ and there exists a sequence of states $r_0, \ldots, r_n \in Q$ such that

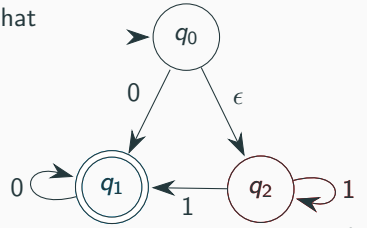
-$r_0 = q_0$,

-$r_{i+1} \in \delta(r_i, w_{i+1})$,
for $0 \leq i \leq n-1$, and

-$r_n \in F$.

Sequence of states:





Regular expressions

Definition (Regular Expressions)
R is a **regular expression** if R is

1. $\{a\}$ for some $a \in \Sigma$,
2. $\{\epsilon\}$,
3. $\varnothing$,
4. $(R_1 \cup R_2)$, where R_1 and R_2 are regular expressions,
5. $(R_1 \circ R_2)$, where R_1 and R_2 are regular expressions,
6. (R_1^*), where R_1 is a regular expression.

The notation R^+ is shorthand for $R \circ R^*$, that is, one or more repetitions of R. Note that $R^+ \cup \{\epsilon\} = R^*$.

Equivalent ways of describing regular languages

Definition (Regular Language)
A language A is **Regular** if there exists a deterministic finite state automaton M such that $L(M) = A$.

Theorem (Equivalence of NFA's and DFA's)
Every non-deterministic finite automaton has an equivalent deterministic finite automaton.

Theorem
A language is regular if and only if some regular expression describes it.

Equivalent ways of describing regular languages

Definition (Regular Language)
A language A is **Regular** if there exists a deterministic finite state automaton M such that $L(M) = A$.

Theorem (Equivalence of NFA's and DFA's)
Every non-deterministic finite automaton has an equivalent deterministic finite automaton.

$\longrightarrow$ *power set construction*

Theorem
A language is regular if and only if some regular expression describes it. $\longrightarrow$ *GNFA's*

Equivalent ways of describing regular languages

Definition (Regular Language)
A language A is **Regular** if there exists a deterministic finite state automaton M such that $L(M) = A$.

Theorem (Equivalence of NFA's and DFA's)
Every non-deterministic finite automaton has an equivalent deterministic finite automaton.

$\longrightarrow$ *power set construction*

Theorem
A language is regular if and only if some regular expression describes it. $\longrightarrow$ *GNFA's*

$\longrightarrow$ We have seen that the following languages are regular.

- $\{w \in \{0,1\}^* : w \text{ is the binary representation of a number which is divisible by } 2\}$
- $\{w \in \{a,b\}^* : w \text{ contains at least two a's}\}$
- $\{w \in \{0,1\}^* : w \text{ contains } 01\}$
- $\{w\}$ for any word w

Closure properties of regular languages

Let A and B be languages over Σ, we define the following operations.

-**Union:** $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$.

-**Intersection:** $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$.

-**Complement:** $\overline{A} = \{x \in \Sigma^* \mid x \notin A\}$.

-**Concatenation:** $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$.

-**Kleene star:** $A^* = \{x_1 \dots x_k \mid k \geq 0 \text{ and, for each } 0 \leq i \leq k, x_i \in A\}$.

Closure properties of regular languages

Let A and B be languages over Σ, we define the following operations.

-**Union:** $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$.

-**Intersection:** $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$.

-**Complement:** $\overline{A} = \{x \in \Sigma^* \mid x \notin A\}$.

-**Concatenation:** $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$.

-**Kleene star:** $A^* = \{x_1 \dots x_k \mid k \geq 0 \text{ and, for each } 0 \leq i \leq k, x_i \in A\}$.

Theorem (Closure properties of regular languages)
Regular languages are closed with respect to

(i) *Union,*

(ii) *Intersection,*

(iii) *Complement,*

(iv) *Concatenation, and*

(v) *Kleene star.*

Closure properties of regular languages

Let A and B be languages over Σ, we define the following operations.

-**Union:** $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$.

-**Intersection:** $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$.

-**Complement:** $\overline{A} = \{x \in \Sigma^* \mid x \notin A\}$.

-**Concatenation:** $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$.

-**Kleene star:** $A^* = \{x_1 \dots x_k \mid k \geq 0 \text{ and, for each } 0 \leq i \leq k, x_i \in A\}$.

Theorem (Closure properties of regular languages)
Regular languages are closed with respect to

(i) *Union,*

(ii) *Intersection,* $\longrightarrow$ *cross-product construction*

(iii) *Complement,*

(iv) *Concatenation, and*

(v) *Kleene star.*

Non-regularity

Theorem (Pumping lemma for regular languages)
If A is a regular language, then there is a number p (the pumping length) where if s is any string in A of length at least p, then s may be divided into three parts, $s = xyz$, satisfying the following conditions:

1. *for each $i \geq 0$, $xy^iz \in A$,*
2. *$|y| > 0$, and*
3. *$|xy| \leq p$.*

All regular languages have the pumping property.
If a language does not have the pumping property then it can't be regular.

Non-regularity

Theorem (Pumping lemma for regular languages)
If A is a regular language, then there is a number p (the pumping length) where if s is any string in A of length at least p, then s may be divided into three parts, $s = xyz$, satisfying the following conditions:

1. *for each $i \geq 0$, $xy^iz \in A$,*
2. *$|y| > 0$, and*
3. *$|xy| \leq p$.*

All regular languages have the pumping property.
If a language does not have the pumping property then it can't be regular.
$\longrightarrow \{0^n1^n \mid 0 \geq n\}$ is not regular.

Non-regularity

Theorem (Pumping lemma for regular languages)
If A is a regular language, then there is a number p (the pumping length) where if s is any string in A of length at least p, then s may be divided into three parts, $s = xyz$, satisfying the following conditions:

1. *for each $i \geq 0$, $xy^iz \in A$,*
2. *$|y| > 0$, and*
3. *$|xy| \leq p$.*

All regular languages have the pumping property.
If a language does not have the pumping property then it can't be regular.
$\longrightarrow \{0^n1^n \mid 0 \geq n\}$ is not regular.

Warning!!
There are non-regular languages which have the pumping property. This is not an **if and only if** statement.

Context-free languages

CFG's

Definition (CFG)
A **context-free grammar** is a 4-tuple (V, Σ, R, S), where

1. V is a finite set called the **variables (non-terminals)**,
2. Σ is a finite set, disjoint from V, called the **terminals**,
3. R is a finite set of **production rules**, with each production rule being a variable and a string of variables and terminals, and
4. $S \in V$ is the **start variable**.

CFG's

Definition (CFG)
A **context-free grammar** is a 4-tuple (V, Σ, R, S), where

1. V is a finite set called the **variables (non-terminals)**,
2. Σ is a finite set, disjoint from V, called the **terminals**,
3. R is a finite set of **production rules**, with each production rule being a variable and a string of variables and terminals, and
4. $S \in V$ is the **start variable**.

Definition (Derivation)
If u, v and w are strings of variables and terminals, and $A \to w$ is a rule of the grammar, we say that uAv **yields** uwv, written $uAv \Rightarrow uwv$.
u **derives** v, written $u \Rightarrow v$, if either $u = v$ or if a sequence $u_1, u_2, \ldots, u_k$ exists such that

$$u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \ldots \Rightarrow u_k \Rightarrow v.$$

Parse tree

Let $G = (V, \Sigma, R, S)$, where $V = \{S, A, B\}$, $\Sigma = \{0, 1, \#\}$ and R is given by

$$S \to A$$

$$A \to 0A1$$

$$A \to B$$

$$B \to \#$$

Parse tree

Let $G = (V, \Sigma, R, S)$, where $V = \{S, A, B\}$, $\Sigma = \{0, 1, \#\}$ and R is given by

$S \to A$

$A \to 0A1$

$A \to B$

$B \to \#$

S

0 0 # 1 1

Parse tree

Let $G = (V, \Sigma, R, S)$, where $V = \{S, A, B\}$, $\Sigma = \{0, 1, \#\}$ and R is given by

$S \to A$

$A \to 0A1$

$A \to B$

$B \to \#$

S
|
A

0 0 # 1 1

Parse tree

Let $G = (V, \Sigma, R, S)$, where $V = \{S, A, B\}$, $\Sigma = \{0, 1, \#\}$ and R is given by

$S \to A$

$A \to 0A1$

$A \to B$

$B \to \#$

0 0 # 1 1

Parse tree

Let $G = (V, \Sigma, R, S)$, where $V = \{S, A, B\}$, $\Sigma = \{0, 1, \#\}$ and R is given by

$S \to A$

$A \to 0A1$

$A \to B$

$B \to \#$

0 0 # 1 1

Parse tree

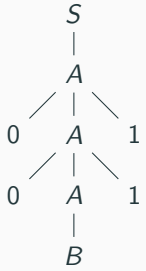
Let $G = (V, \Sigma, R, S)$, where $V = \{S, A, B\}$, $\Sigma = \{0, 1, \#\}$ and R is given by

$S \to A$

$A \to 0A1$

$A \to B$

$B \to \#$



0 0 # 1 1

Parse tree

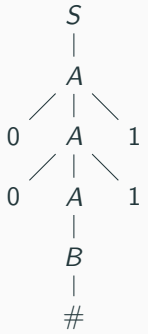
Let $G = (V, \Sigma, R, S)$, where $V = \{S, A, B\}$, $\Sigma = \{0, 1, \#\}$ and R is given by

$S \to A$

$A \to 0A1$

$A \to B$

$B \to \#$



0 0 # 1 1

PDA's

Definition (PDA)
A **Pushdown automata** is a 6-tuple $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$, where

-Q is a finite set of states,

-Σ is the finite input alphabet,

-Γ is the finite stack alphabet,

-$\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to \mathcal{P}(Q \times \Gamma_\epsilon)$ is the transition function,

-$q_0 \in Q$ is the start state, and

-$F \subseteq Q$ is the set of accept states.

PDA's

Definition (PDA)
A **Pushdown automata** is a 6-tuple $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$, where
-Q is a finite set of states,
-Σ is the finite input alphabet,
-Γ is the finite stack alphabet,
-$\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to \mathcal{P}(Q \times \Gamma_\epsilon)$ is the transition function,
-$q_0 \in Q$ is the start state, and
-$F \subseteq Q$ is the set of accept states.

Definition
The automaton P **accepts** $w = w_1 \dots w_n \in \Sigma^*$ if a sequence of states $r_0, \dots, r_n \in Q$ and a sequence of strings $s_0, \dots, s_n \in \Gamma^*$ exists such that
-$r_0 = q_0$ and $s_0 = \epsilon$,
-for $0 \leq i < n$ we have $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$ where
$s_i = at$ and $s_{i+1} = bt$ for some $a, b \in \Gamma_\epsilon$ and $t \in \Gamma^*$, and
-$r_n \in F$.

Equivalent ways of descibing context-free languages

Definition
A language is A is **context-free** if there exists a context-free grammar G such that $L(G) = A$.

Theorem
Any context-free language can be generated by a context-free grammar in Chomsky Normal Form (CNF).

Theorem
A language is context-free if and only if some pushdown automata recognises it.

Equivalent ways of descibing context-free languages

Definition
A language is A is **context-free** if there exists a context-free grammar G such that $L(G) = A$.

Theorem
Any context-free language can be generated by a context-free grammar in Chomsky Normal Form (CNF).

Theorem
A language is context-free if and only if some pushdown automata recognises it.

$\longrightarrow$ *Simulate the derivation on the stack.*

$\longrightarrow$ *Recursive construction simulating each transition.*

Equivalent ways of descibing context-free languages

Definition
A language is A is **context-free** if there exists a context-free grammar G such that $L(G) = A$.

Theorem
Any context-free language can be generated by a context-free grammar in Chomsky Normal Form (CNF).

Theorem
A language is context-free if and only if some pushdown automata recognises it.

$\longrightarrow$ *Simulate the derivation on the stack.*

$\longrightarrow$ *Recursive construction simulating each transition.*

$\longrightarrow$ The following languages are context-free.

$\{w \in \{(,)\} : w \text{ is a well parenthesized expression}\}$

if-then-else language

$\{0^n 1^n : n \geq 0\}$

Non-context-free languages

Theorem (Pumping lemma for context-free languages)
If A is a context-free language, then there is a number p (the pumping length) where, if s is any string in A of length at least p, then s may be divided into five parts, $s = uvxyz$, satisfying the following conditions:

1. *for each $i \geq 0$, $uv^ixy^iz \in A$,*
2. *$|vy| > 0$, and*
3. *$|vxy| \leq p$.*

All context-free languages have the context-free pumping property.
If a language does not have the context-free pumping property then it can't be context-free.

Non-context-free languages

Theorem (Pumping lemma for context-free languages)
If A is a context-free language, then there is a number p (the pumping length) where, if s is any string in A of length at least p, then s may be divided into five parts, $s = uvxyz$, satisfying the following conditions:

1. *for each $i \geq 0$, $uv^i xy^i z \in A$,*
2. *$|vy| > 0$, and*
3. *$|vxy| \leq p$.*

All context-free languages have the context-free pumping property.
If a language does not have the context-free pumping property then it can't be context-free.
$\longrightarrow \{a^n b^n c^n \mid n \geq 0\}$ is not context-free.

Closure properties of context-free languages

Theorem (Closure properties of context-free languages)
Context-free languages are closed with respect to

(i) *Union,*

(ii) *Intersection with a regular language,*

(iii) *Concatenation, and*

(iv) *Kleene star.*

Closure properties of context-free languages

Theorem (Closure properties of context-free languages)
Context-free languages are closed with respect to

(i) *Union,*

(ii) *Intersection with a regular language,*

(iii) *Concatenation, and*

(iv) *Kleene star.*

Context-free languages are not closed with respect to

(I) *Intersection, and* $\longrightarrow \{a^n b^n c^n \mid n \geq 0\} = \{a^n b^n c^m \mid m, n \geq 0\} \cap \{a^m b^n c^n \mid m, n \geq 0\}$

(II) *Complement.* $\longrightarrow$ *use that* $A \cap B = \overline{\overline{A} \cup \overline{B}}$

Turing machines

Definition (TM)
A **Turing machine** is a 7-tuple $T = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where

1. Q is the set of states,
2. Σ is the input alphabet not containing the blank symbol $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the transition function,
5. $q_0 \in Q$ is the start state,
6. $q_{\text{accept}} \in Q$ is the accept state, and
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{accept}} \neq q_{\text{reject}}$.

Computation

For strings $u, v \in \Gamma^*$ and $q \in Q$ we write $u\,q\,v$ for the **configuration** where q is the current state, $uv\sqcup\sqcup\ldots$ is the content of the tape and the head is positioned over the first symbol of v.

Computation

For strings $u, v \in \Gamma^*$ and $q \in Q$ we write $u\,q\,v$ for the **configuration** where q is the current state, $uv\sqcup\sqcup\ldots$ is the content of the tape and the head is positioned over the first symbol of v.

Definition (Computation of TM's)
Let $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ be a Turing machine and let $w \in \Sigma^*$ a string. The TM M **accepts** (**rejects**) w if a sequence of configurations $C_1, \ldots, C_k$ exists such that
- $C_1 = q_0\,w$ is the starting configuration,
- for $1 \leq i < k$ we have that C_i yields C_{i+1}, and
- $Q_k = u\,q_{\text{accept}}\,v$ ($Q_k = u\,q_{\text{reject}}\,v$) for some $u, v \in \Gamma^*$ is an accepting (rejecting) configuration.

Recognizing vs. deciding

Definition
A language L is **recursive enumerable** or **Turing recognizable** if it is recognized by some Turing machine, that is there is a Turing machine M such that $L(M) = L$.

Definition
A language L is **recursive** or **Turing decidable** if it is decided by some Turing machine, that is there is a Turing machine M such that $L(M) = L$ and M halts on every input string.

Recognizing vs. deciding

Definition
A language L is **recursive enumerable** or **Turing recognizable** if it is recognized by some Turing machine, that is there is a Turing machine M such that $L(M) = L$.

Definition
A language L is **recursive** or **Turing decidable** if it is decided by some Turing machine, that is there is a Turing machine M such that $L(M) = L$ and M halts on every input string.

$\longrightarrow$ The following languages are Turing decidable.

$$\{w\#w \mid w \in \{0,1\}^*\}$$
$$\{a^n b^n c^n \mid n \geq 0\}$$
$$\{0^{2^n} \mid n \geq 0\}$$

Equivalent models of TM's

Theorem
Every multitape Turing machine has an equivalent single tape Turing machine.

Theorem
Every non-deterministic Turing machine has an equivalent deterministic Turing machine.

Equivalent models of TM's

Theorem
Every multitape Turing machine has an equivalent single tape Turing machine.

$\longrightarrow$ *Simulate both tapes on one tape using two $\bullet$ to mark where both heads are.*

Theorem
Every non-deterministic Turing machine has an equivalent deterministic Turing machine.

$\longrightarrow$ *For each node t of tree of computation, simulate the deterministic computation up to t.*

Equivalent models of TM's

Theorem
Every multitape Turing machine has an equivalent single tape Turing machine.
$\longrightarrow$ *Simulate both tapes on one tape using two • to mark where both heads are.*

Theorem
Every non-deterministic Turing machine has an equivalent deterministic Turing machine.
$\longrightarrow$ *For each node t of tree of computation, simulate the deterministic computation up to t.*

Informal: Every sensible model of computation that allows arbitrary access to memory and each step of computation performs only finite work will be equivalent to a TM.

Informal: Everything an algorithm can compute can be computed by a TM (Church-Turing thesis says that λ-calculus is equivalent to Turing machines).

Consequence: The formalization of the term algorithm allows us to prove that there are problems for which there is no algorithm (they are not Turing decidable).

Languages related to regular languages

Theorem
The following languages are Turing decidable.

$$A_{\text{DFA}} = \{\langle A, w\rangle \mid A \text{ is a DFA that accepts } w\};$$
$$A_{\text{NFA}} = \{\langle B, w\rangle \mid B \text{ is an NFA that accepts } w\};$$
$$E_{\text{DFA}} = \{\langle A\rangle \mid A \text{ is a DFA and } L(A) = \emptyset\};$$
$$EQ_{\text{DFA}} = \{\langle A, B\rangle \mid A, B \text{ are DFA's and } L(A) = L(B)\};$$
$$A_{\text{CFG}} = \{\langle G, w\rangle \mid G \text{ is a CFG and } w \in L(G)\};$$
$$E_{\text{CFG}} = \{\langle G\rangle \mid G \text{ is a CFG and } L(G) = \emptyset\}.$$

Theorem
Every context-free language A is Turing decidable.

Existence of undecidable and unrecognizable languages

We proved the following theorem by a counting argument using diagonalization.

Theorem
There exists a language which is not Turing recognizable.

Theorem
The language

$$A_{\mathrm{TM}} = \{\langle A, w\rangle \mid A \text{ is a TM that accepts } w\}$$

is not Turing decidable.

Theorem
$\overline{A_{\mathrm{TM}}}$ *is not Turing recognizable.*

Existence of undecidable and unrecognizable languages

We proved the following theorem by a counting argument using diagonalization.

Theorem
There exists a language which is not Turing recognizable. $\longrightarrow$ *diagonalization*

Theorem
The language

$$A_{\mathrm{TM}} = \{\langle A, w\rangle \mid A \text{ is a TM that accepts } w\}$$

is not Turing decidable. $\longrightarrow$ *diagonalization*

Theorem
$\overline{A_{\mathrm{TM}}}$ *is not Turing recognizable.*